Kernix benchmark report: 0.1.1

Measured 2026-10-19T14:28:29Z on 12th Gen Intel(R) Core(TM) i3-1215U.

Rust: rustc 1.99.0 (b940084d7 2026-09-28); NumPy: 2.5.4; one CPU thread; float64.

Source: `b208e4231d00635eb4abb5c1647222f5779c43a9`; working tree dirty: True.

Comparative CPU results

Workload	Kernix p50 / p95 (us)	ndarray p50 / p95 (us)	NumPy p50 / p95 (us)
add_65536_f64	24.624 / 31.236	21.962 / 30.357	23.283 / 23.751
matmul_128_f64	3311.847 / 3468.955	88.635 / 152.673	1174.340 / 1224.260
matmul_32_f64	35.003 / 63.825	1.546 / 1.589	15.601 / 22.599
softmax_256x64_f64	765.891 / 784.432	72.548 / 110.273	211.214 / 218.471
sum_65536_f64	37.352 / 58.822	6.328 / 8.713	12.246 / 16.379

Method and limits

All operations allocate their output except scalar sums. Inputs are preallocated and contiguous. ndarray uses its default matrixmultiply engine (no BLAS feature); NumPy uses the build recorded in raw.json. BLAS/OMP thread counts are fixed to one. These are warmed, sequential wall-clock samples on a shared laptop; CPU frequency and other system load are uncontrolled. p95 is the maximum of 15 batch means, not a per-request tail-latency guarantee. Different sum orders may round differently. The ndarray softmax is an explicitly coded stable rowwise comparison, not a native ndarray softmax API. NumPy timings include Python call overhead. No GPU comparison. No claim that a complete Full HD model reaches 120 FPS follows from these measurements. raw.json contains all measurements, dependencies and environment metadata.